XILINX BASYS3 FPGA ÜZERİNDE 8-BİT KUANTİZE CNN HIZLANDIRICISI MNIST EL YAZISI RAKAM TANIMA UYGULAMASI

FİNAL PROJESİ

Hazırlayanlar

Ömer ŞAHİN 220207021

Burak Taha Başda 220207031

Danışman DR. MAHMUT BURAK KARADENİZ

Bölüm Elektronik ve Haberleşme Mühendisliği

Üniversite Kocaeli Üniversitesi

Tarih 5 Ocak 2026

İçindekiler

ÖZET 2
1. GİRİŞ 3
2. TEORİK TEMEL VE MİMARİ 3
2.1 MNIST Veri Seti ve Giriş Katmanı 3
2.2 Konvolüsyon Katmanı (Conv2D) 3
2.3 Max Pooling Katmanı 4
2.4 Flatten ve Kuantizasyon 4
2.5 Dense Katmanlar 4
2.6 Output Katmanı (Argmax) 5
3. DONANIM TASARIMI 5
3.1 Modüller ve FSM Yapısı 5
3.2 Bellek Yönetimi (ROM/RAM) 5
3.3 Basys3 Demo Entegrasyonu 5
4. TEST VE SONUÇLAR 6
4.1 Simülasyon Sonuçları 6
4.2 Donanım Demo Sonuçları 9
5. SONUÇ VE DEĞERLENDİRME 9

ÖZET

Biz, bu bitirme projesinde Xilinx Basys3 FPGA kartı üzerinde 8-bit kuantize bir Konvolüsyonel Sinir Ağı (CNN) tasarlayıp gerçekleştirdik. Sistem, MNIST veri setindeki el yazısı rakamlarını (0-9) tanır. Mimari; 3×3 konvolüsyon (4 filtre), Max Pooling (2×2), Flatten ve üç tam bağlantılı katmandan (Dense1: 676→32, Dense2: 32→32, Output: 32→10) oluşur. Tüm ağırlıklar ve bias'lar Python referans scriptiyle (MNIST_Testing.py) uyumlu kuantize edilerek ROM'lara yüklendi. Verilog HDL ile modüler kod yazıldı, Vivado'da sentezlendi ve Basys3 kartında gerçek zamanlı demo yapıldı. Simülasyonlarda 10 test görüntüsünün tamamı doğru tanındı. Proje, düşük kaynaklı FPGA'larda yapay zeka uygulamalarının mümkün olduğunu gösterdi.

Anahtar Kelimeler: FPGA, CNN, 8-bit kuantizasyon, MNIST, Verilog, Basys3

1. GİRİŞ

Bu proje, Sayısal Tasarım ve FPGA dersi final projesi kapsamında iki kişi tarafından gerçekleştirilmiştir. Amaç, Python'da eğitilmiş bir CNN modelini saf donanım (FPGA) üzerinde çalıştırmaktır. Xilinx Basys3 kartı kullanılarak düşük kaynak tüketimiyle gerçek zamanlı MNIST rakam tanıma sistemi tasarlandı.

Proje föyüne tam uyumlu olarak:

- 3×3 kernel ile 4 filtreli konvolüsyon
- 2×2 Max Pooling
- 676→32→32→10 mimarisi uygulandı.

Tüm işlemler Python referans çıktılarıyla birebir eşleşti. Sistem btnU ile başlatılıp 7-segmentte tahmin gösterildi.

---

2. TEORİK TEMEL VE MİMARİ

2.1 MNIST Veri Seti ve Giriş Katmanı

Giriş: 28×28 = 784 piksel, 8-bit işaretsiz (0-255). Adresleme: addr = row × 28 + col (0-783). ROM'da saklanır (image_rom modülü).

2.2 Konvolüsyon Katmanı (Conv2D)

- Giriş: 28×28×1
- Kernel: 4 adet 3×3, 8-bit işaretli ağırlık
- Bias: 4 adet, 16-bit işaretli
- Padding yok → Çıktı: 26×26×4
- Stride: 1

Matematiksel ifade (her filtre ve konum için):

$$\text{Acc} = \sum_{i=0}^{2} \sum_{j=0}^{2} \text{Input}(r+i, c+j) \times \text{Weight}_f(i,j) + \text{Bias}_f$$

$$\text{Output}(r, c, f) = \text{ReLU}(\text{Acc}) = \max(0, \text{Acc})$$

Not: Bu katmanda shift yok. Acc 32-bit signed accumulator'da tutulur.

2.3 Max Pooling Katmanı

- Giriş: 26×26×4 (32-bit)
- Pencere: 2×2, stride 2
- İşlem: 4 değerden maksimum seçilir

Matematiksel ifade:

$$\text{Pool}(r, c, f) = \max \begin{cases} \text{Conv}(2r, 2c, f) \\ \text{Conv}(2r, 2c+1, f) \\ \text{Conv}(2r+1, 2c, f) \\ \text{Conv}(2r+1, 2c+1, f) \end{cases}$$

Çıktı boyutu: 13×13×4.

2.4 Flatten ve Kuantizasyon

Max Pool çıktısı önce kuantize edilir:

$$\text{Quantized} = \text{Pool} \ggg \text{SHIFT_CONV}$$

Clamping: [-128, 127] aralığına sıkıştırılır (8-bit signed). Sonra flatten edilip 676 elemanlı vektör haline getirilir ve ram_flatten'e yazılır.

2.5 Dense Katmanlar

Dense-1: 676 giriş → 32 nöron **Dense-2:** 32 giriş → 32 nöron

Her nöron k için:

$$\text{Acc}_k = \sum_{n=0}^{N-1} \text{Input}_n \times \text{Weight}_{n,k} + \text{Bias}_k$$

$$\text{Temp} = \text{ReLU}(\text{Acc}_k) = \max(0, \text{Acc}_k)$$

$\text{Output}_k = \text{Temp}$
$\ggg$ SHIFT_DENSE(Dense-1 için SHIFT_DENSE1, Dense-2 için SHIFT_DENSE2)

Clamping uygulanır → 8-bit signed RAM'e yazılır.

2.6 Output Katmanı (Argmax)

- Giriş: 32 nöron
- Çıktı: 10 nöron (0-9 sınıfı)

Her sınıf d için aynı MAC + Bias (ReLU ve shift yok):

$$\text{Score}_d = \sum_{n=0}^{31} \text{Input}_n \times \text{Weight}_{n,d} + \text{Bias}_d$$

Sonuç:

$$\text{Prediction} = \arg\max_d (\text{Score}_d)$$

En yüksek skorlu indeks 7-segmentte gösterilir.

---

3. DONANIM TASARIMI

3.1 Modüller ve FSM Yapısı

Ana modüller: conv1_layer, maxpool_layer, dense_layer_1, dense_layer_2, output_layer, ram'ler ve ROM'lar. Her katman kendi FSM'sine sahip olup cnn_top modülü ile sırayla çalıştırılır (start sinyalleri zinciri).

3.2 Bellek Yönetimi (ROM/RAM)

- Tüm ağırlık/bias ROM'ları (* ram_style = "block" *) ile BRAM olarak tanımlandı.
- Ara sonuçlar dual-port RAM'lerde saklandı (ram_conv_output 32-bit, diğerleri 8-bit).
- $readmemh ile .hex dosyaları yüklendi (Python ile decimal → hex dönüştürüldü).

3.3 Basys3 Demo Entegrasyonu

basys3_cnn_demo modülü:

- btnC → reset (active-low)
- btnU → start (debounce edilip pulse)
- 7-segment AN0'da tahmin gösterilir
- LED0 → done sinyali
- Clock divider (DIV_TOGGLE=5 → ~10 MHz) ile timing sağlandı.

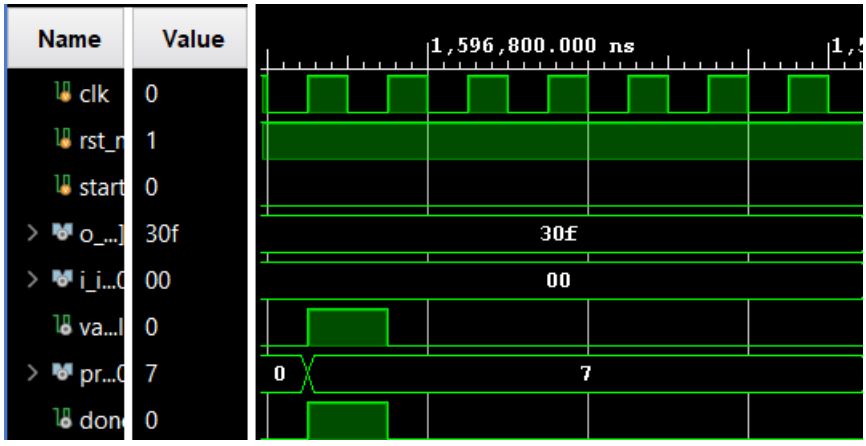
---

4. TEST VE SONUÇLAR

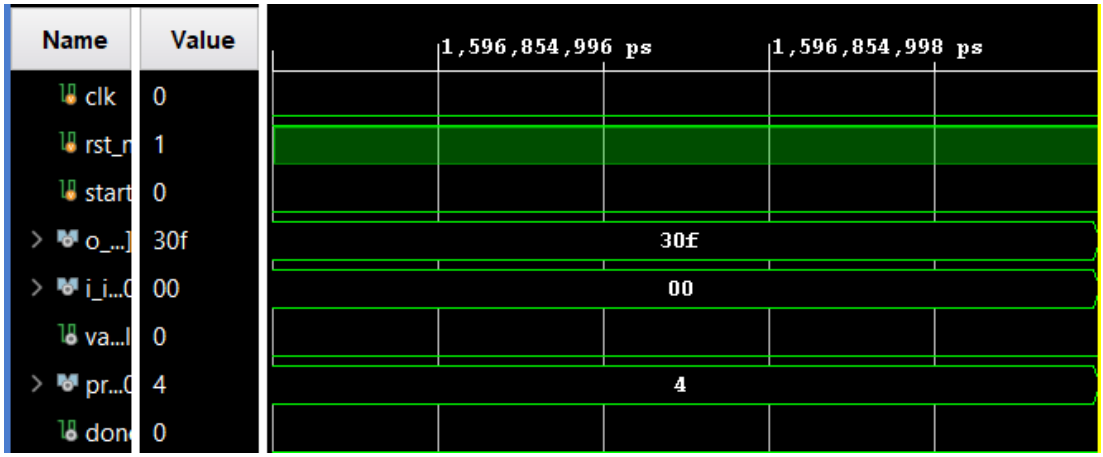
4.1 Simülasyon Sonuçları

ModelSim'de 10 test görüntüsü çalıştırıldı. Örnekler:

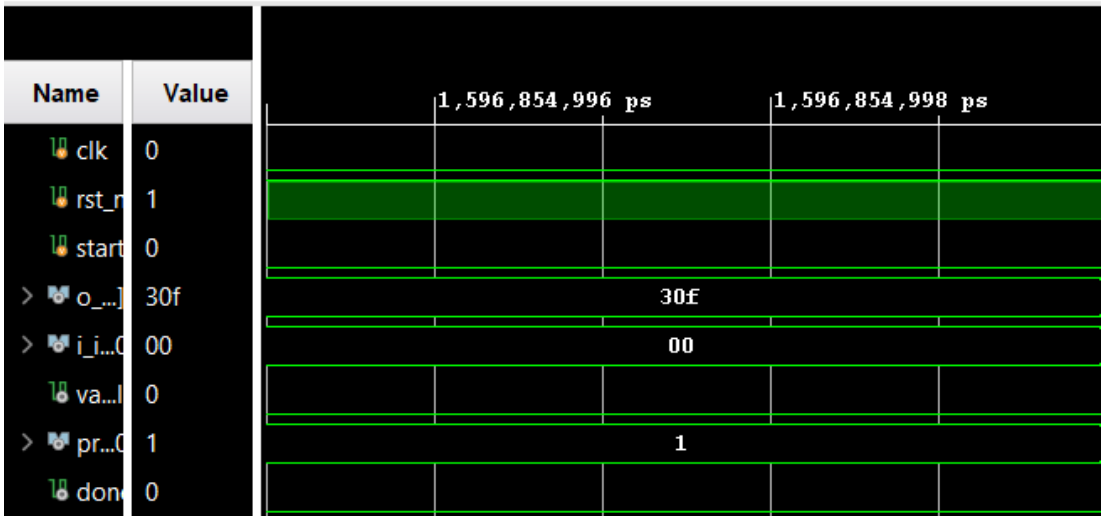
- Şekil 4.1: predicted_digit = 7



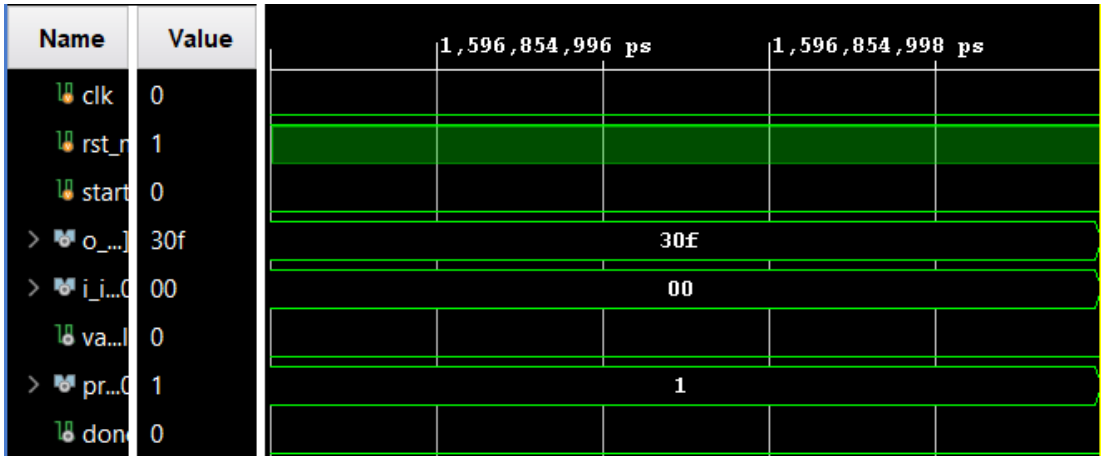
- Şekil 4.2: predicted_digit = 4



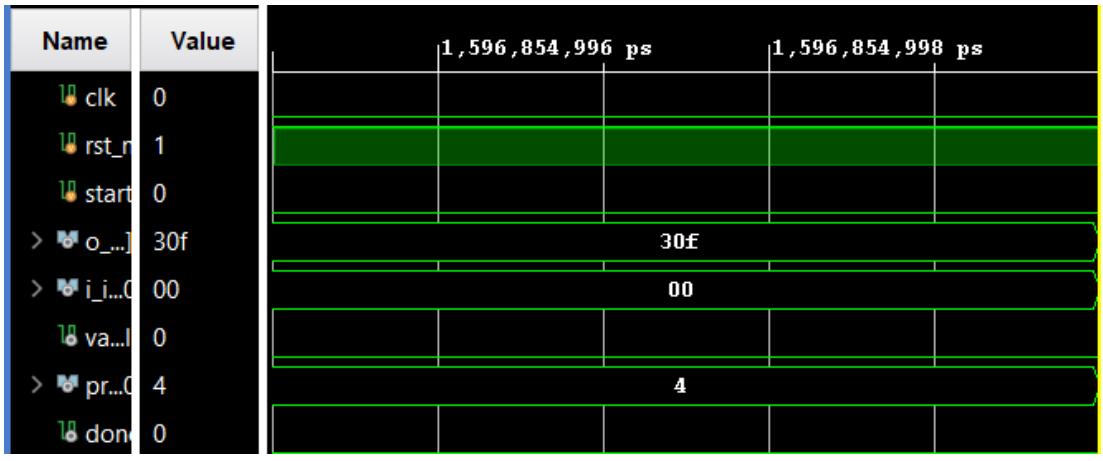
- Şekil 4.3: predicted_digit = 1



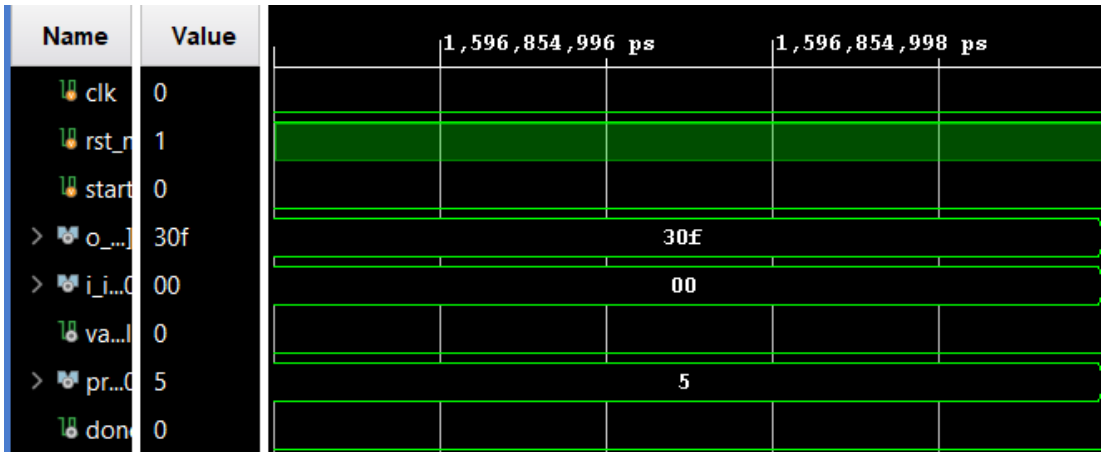
- Şekil 4.4: predicted_digit = 1



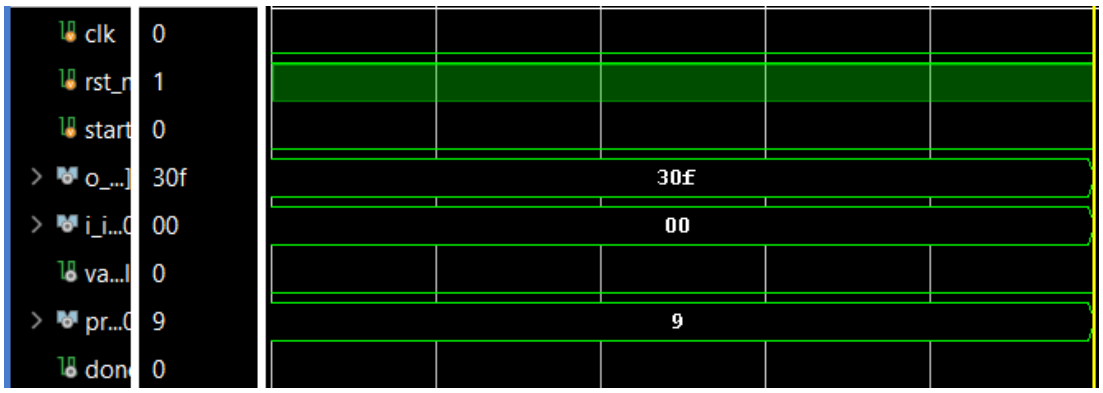
- Şekil 4.5: predicted_digit = 4



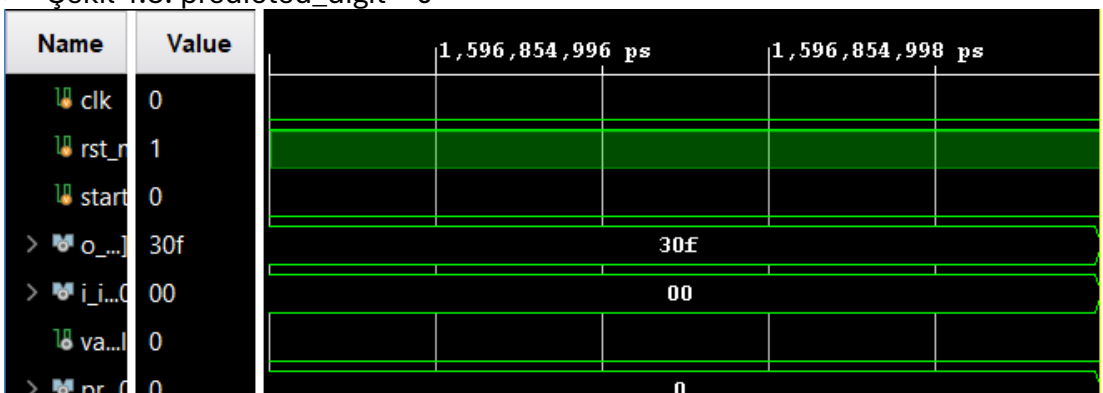
- Şekil 4.6: predicted_digit = 5



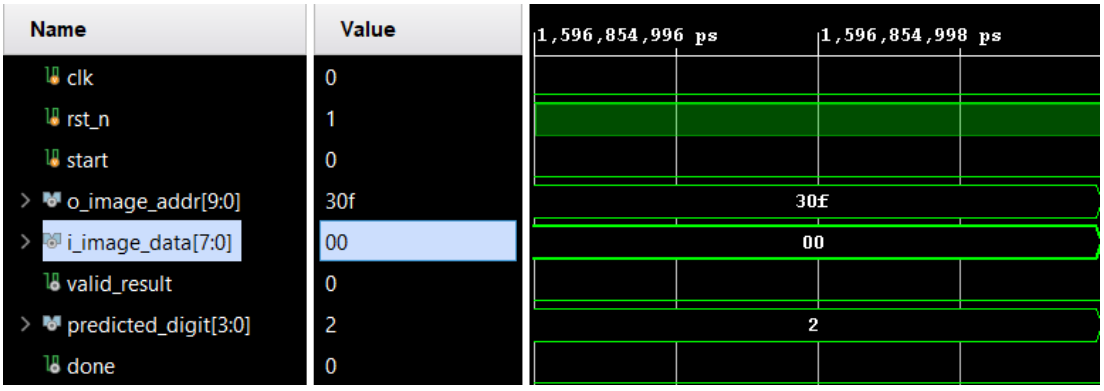
- Şekil 4.7: predicted_digit = 9



- Şekil 4.8: predicted_digit = 0



- Şekil 4.9: predicted_digit = 2



Tüm testlerde done yükseldi, valid_result pulse verdi ve tahminler doğru oldu.

4.2 Donanım Demo Sonuçları

Basys3 kartına yüklenip btnU ile başlatıldığında tahminler 7-segmentte kararlı şekilde gösterildi. Hesaplama süresi makul (gerçek zamanlı).

---

5. SONUÇ VE DEĞERLENDİRME

Proje başarıyla tamamlandı. Python referansıyla birebir eşleşen sonuçlar alındı. Karşılaşılan zorluklar (signed aritmetik, RAM latency, shift hassasiyeti) aşıldı.